

Time Complexity Analysis

1 Data Structures

1.1 HashTable

Insert Operation

```
bool HashTable::insert(const char *key, void *value) {
    if (shouldResize()) {
        rehashInternal(capacity * 2 + 1);
    }
    int hash = hashKey(key);
    for (int attempt = 0; attempt < capacity; ++attempt) {
        int index = probeIndex(hash, attempt);
        if (!table[index].occupied || table[index].deleted) {
            snprintf(table[index].key, sizeof(table[index].key), "%s", key);
            table[index].value = value;
            table[index].occupied = true;
            table[index].deleted = false;
            count++;
            return true;
        }
        if (!table[index].deleted &&
            strcmp(table[index].key, key) == 0) {
            table[index].value = value;
            return true;
        }
    }
    return false;
}
```

- `hashKey()`: iterates through key string of length k : $O(k)$
- For loop with linear probing:
 - Average case: few probes needed $\rightarrow O(1)$
 - Worst case: all slots probed $\rightarrow O(n)$
- String copy and assignments: $O(k)$

Calculation:

$$T_{\text{avg}}(n) = O(k) + O(1) + O(k) = O(1) \text{ (assuming constant } k)$$
$$T_{\text{worst}}(n) = O(k) + O(n) + O(k) = O(n)$$

Find Operation

```
void *HashTable::find(const char *key) const {
    int slot = findSlot(key);
    if (slot == -1)
        return nullptr;
    return table[slot].value;
}

int HashTable::findSlot(const char *key) const {
    int hash = hashKey(key);
    for (int attempt = 0; attempt < capacity; ++attempt) {
        int index = probeIndex(hash, attempt);
        if (!table[index].occupied && !table[index].deleted) {
            return -1;
        }
        if (table[index].occupied && !table[index].deleted &&
            strcmp(table[index].key, key) == 0) {
            return index;
        }
    }
    return -1;
}
```

- `hashKey()`: $O(k)$
- For loop probing:
 - Average case: $O(1)$
 - Worst case: $O(n)$
- `strcmp()`: $O(k)$ per comparison

Calculation:

$$T_{\text{avg}}(n) = O(k) + O(1) = O(1) \text{ (assuming constant } k)$$
$$T_{\text{worst}}(n) = O(k) + \sum_{i=1}^n O(k) = O(k) + n \cdot O(k) = O(nk) = O(n)$$

Rehash Operation

```

void HashTable::rehashInternal(int newCapacity) {
    Entry *oldTable = table;
    int oldCapacity = capacity;
    capacity = newCapacity;
    table = new Entry[capacity];
    count = 0;
    for (int i = 0; i < oldCapacity; ++i) {
        if (oldTable[i].occupied && !oldTable[i].deleted) {
            insert(oldTable[i].key, oldTable[i].value);
        }
    }
    delete[] oldTable;
}

```

- Allocate new table: $O(n_{\text{new}})$
- For loop iterates n_{old} times
- Each insert: $O(1)$ average

Calculation:

$$\begin{aligned}
 T(n) &= O(n_{\text{new}}) + \sum_{i=1}^{n_{\text{old}}} O(1) \\
 &= O(n_{\text{new}}) + O(n_{\text{old}}) \\
 &= O(n)
 \end{aligned}$$

1.2 MinHeap

Insert Operation

```

void MinHeap<T>::insert(const T &key) {
    if (isFull())
        resize();
    arr[++N] = key;
    upheap(last());
}

void MinHeap<T>::upheap(int i) {
    T temp = std::move(arr[i]);
    while (i > 0 && arr[parent(i)] > temp) {
        arr[i] = std::move(arr[parent(i)]);
        i = parent(i);
    }
    arr[i] = std::move(temp);
}

```

- Insert at end: $O(1)$
- `upheap()`: traverses from leaf to root

- Height of heap: $h = \lfloor \log_2 n \rfloor$
- Each level: $O(1)$ comparison and swap

Calculation:

$$\begin{aligned}
 T(n) &= O(1) + \sum_{i=0}^h O(1) \\
 &= O(1) + (h + 1) \cdot O(1) \\
 &= O(1) + O(\log n) \\
 &= O(\log n)
 \end{aligned}$$

Extract Min Operation

```

T MinHeap<T>::extractMin() {
    if (isEmpty())
        return T();
    if (N == 1) {
        N--;
        return arr[1];
    }
    T root = move(arr[1]);
    arr[1] = move(arr[last()]);
    N--;
    downheap(1);
    return root;
}

void MinHeap<T>::downheap(int i) {
    T temp = std::move(arr[i]);
    int j = left(i);
    while (j <= N) {
        int r = j + 1;
        if (r <= N && arr[j] > arr[r])
            j = r;
        if (temp <= arr[j])
            break;
        arr[i] = std::move(arr[j]);
        i = j;
        j = left(i);
    }
    arr[i] = std::move(temp);
}

```

- Swap root with last element: $O(1)$
- `downheap()`: traverses from root to leaf
 - Height of heap: $h = \lfloor \log_2 n \rfloor$

- Each level: $O(1)$ comparison and swap

Calculation:

$$\begin{aligned}
 T(n) &= O(1) + \sum_{i=0}^h O(1) \\
 &= O(1) + (h + 1) \cdot O(1) \\
 &= O(1) + O(\log n) \\
 &= O(\log n)
 \end{aligned}$$

Find Index Operation

```
int MinHeap<T>::findIndex(const T &key) const {
    for (int i = 1; i <= N; ++i) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
```

- Linear search through all elements
- Each comparison: $O(1)$

Calculation:

$$T(n) = \sum_{i=1}^n O(1) = n \cdot O(1) = O(n)$$

Decrease Key Operation

```
void MinHeap<T>::decreaseKey(int i, const T &newValue) {
    if (newValue > arr[i])
        return;
    arr[i] = newValue;
    upheap(i);
}
```

- Assignment: $O(1)$
- upheap(): $O(\log n)$

Calculation:

$$T(n) = O(1) + O(\log n) = O(\log n)$$

1.3 Graph (Adjacency List)

Let V = number of vertices, E = number of edges.

Add Vertex Operation

```
void Graph::addVertex(const char *name) {
    if (findVertexNode(name))
        return;
    VertexNode *newVertex = new VertexNode(name);
    newVertex->next = head;
    head = newVertex;
    vertexLookup.insert(name, newVertex);
}
```

- `findVertexNode()`: hash table lookup $\rightarrow O(1)$ avg
- Allocate new vertex: $O(1)$
- Insert at head of list: $O(1)$
- Hash table insert: $O(1)$ avg

Calculation:

$$T(V) = O(1) + O(1) + O(1) + O(1) = O(1) \text{ average}$$

Add Edge Operation

```
void Graph::addEdge(const char *from, const char *to, int w) {
    VertexNode *fromVertex = findVertexNode(from);
    if (!fromVertex) {
        addVertex(from);
        fromVertex = findVertexNode(from);
    }
    VertexNode *toVertex = findVertexNode(to);
    if (!toVertex) {
        addVertex(to);
        toVertex = findVertexNode(to);
    }
    EdgeNode *newEdge1 = new EdgeNode(to, w);
    if (!fromVertex->adjHead)
        fromVertex->adjHead = newEdge1;
    else {
        EdgeNode *temp = fromVertex->adjHead;
        while (temp->next)
            temp = temp->next;
        temp->next = newEdge1;
    }
    // Same for reverse edge (undirected graph)
    ...
}
```

- Find/add vertices: $O(1)$ avg each
- Create edge node: $O(1)$
- Traverse to end of adjacency list: $O(d)$ where $d = \text{degree of vertex}$
- Repeat for reverse edge: $O(d')$

Calculation:

$$\begin{aligned} T(V, E) &= O(1) + O(1) + O(1) + O(d) + O(d') \\ &= O(d) \end{aligned}$$

Clear Operation

```
void Graph::clear() {
    VertexNode *currVertex = head;
    while (currVertex) {
        EdgeNode *currEdge = currVertex->adjHead;
        while (currEdge) {
            EdgeNode *nextEdge = currEdge->next;
            delete currEdge;
            currEdge = nextEdge;
        }
        VertexNode *nextVertex = currVertex->next;
        delete currVertex;
        currVertex = nextVertex;
    }
    head = nullptr;
    vertexLookup.clear();
}
```

- Outer loop: iterates V times
- Inner loop: for each vertex, iterates through all its edges
- Total edges across all vertices: $2E$ (undirected graph)

Calculation:

$$\begin{aligned} T(V, E) &= \sum_{v=1}^V \left(O(1) + \sum_{e=1}^{d_v} O(1) \right) \\ &= \sum_{v=1}^V O(1) + \sum_{v=1}^V O(d_v) \\ &= O(V) + O(2E) \\ &= O(V + E) \end{aligned}$$

2 Core Algorithms

2.1 Dijkstra's Algorithm (ShortestPath::compute)

Step 1: Initialization

```
VertexNode **vertices = new VertexNode *[numNodes];
int *indexStorage = new int[numNodes];
HashTable indexMap(numNodes * 2 + 1);

VertexNode *curr = graph.getHead();
int idx = 0;
while (curr && idx < numNodes) {
    vertices[idx] = curr;
    indexStorage[idx] = idx;
    indexMap.insert(curr->name, &indexStorage[idx]);
    curr = curr->next;
    idx++;
}
```

- Allocating `vertices` array: $O(V)$
- Allocating `indexStorage` array: $O(V)$
- Creating hash table: $O(1)$
- While loop iterates V times, each iteration:
 - Assignment operations: $O(1)$
 - Hash table insert: $O(1)$ average
- While loop total: $\sum_{i=1}^V O(1) = O(V)$

Step 1 Total: $O(V) + O(V) + O(1) + O(V) = O(V)$

Step 2: Array Initialization

```
float *distances = new float[numNodes];
bool *visited = new bool[numNodes];
bool *inHeap = new bool[numNodes];
int *previous = new int[numNodes];

for (int i = 0; i < numNodes; ++i) {
    distances[i] = INF;
    visited[i] = false;
    inHeap[i] = false;
    previous[i] = -1;
}
```

- Allocating 4 arrays: $4 \times O(V) = O(V)$

- For loop iterates V times, each iteration: $O(1)$
- For loop total: $\sum_{i=1}^V O(1) = O(V)$

Step 2 Total: $O(V) + O(V) = O(V)$

Step 3: Main Loop with Extraction

```
while (!heap.isEmpty()) {
    HeapNode node = heap.extractMin();
    int u = node.vertexIndex;

    if (u < 0 || visited[u])
        continue;

    visited[u] = true;
    inHeap[u] = false;
    if (u == destIndex)
        break;
    ...
}
```

- Loop executes at most V times for visited vertices (each vertex processed once due to **visited** check)
- Each `extractMin()`: $O(\log V)$
- Check and assignment operations: $O(1)$

Step 3 Total: $\sum_{i=1}^V O(\log V) = V \cdot O(\log V) = O(V \log V)$

Step 4: Edge Relaxation

```
EdgeNode *edge = vertices[u]->adjHead;
while (edge) {
    int v = getIndex(indexMap, edge->to);
    if (v != -1 && !visited[v]) {
        float newDist = distances[u] + edge->weight;
        if (newDist < distances[v]) {
            distances[v] = newDist;
            previous[v] = u;
            if (inHeap[v]) {
                int position = heap.findIndex(HeapNode(v, newDist));
                heap.decreaseKey(position, HeapNode(v, newDist));
            } else {
                heap.insert(HeapNode(v, newDist));
                inHeap[v] = true;
            }
        }
    }
    edge = edge->next;
}
```

- This runs for each edge in the adjacency list: $2E$ times total (undirected graph stores each edge twice)
- For each edge:
 - `getIndex()` hash table lookup: $O(1)$ average
 - Distance calculation and comparison: $O(1)$
 - `heap.findIndex()`: $O(V)$ (linear search through heap)
 - `heap.decreaseKey()`: $O(\log V)$
 - Or `heap.insert()`: $O(\log V)$
- Per edge: $O(1) + O(1) + O(V) + O(\log V) = O(V)$

Step 4 Total: $\sum_{i=1}^{2E} O(V) = 2E \cdot O(V) = O(EV)$

Step 5: Path Reconstruction

```
int pathLength = 0;
for (int v = destIndex; v != -1; v = previous[v])
    pathLength++;

result.nodes = new char *[pathLength];
result.nodeCount = pathLength;
int pos = pathLength - 1;
for (int v = destIndex; v != -1; v = previous[v]) {
    result.nodes[pos] = new char[64];
    snprintf(result.nodes[pos], 64, "%s", vertices[v]->name);
    pos--;
}
```

- First loop counts path length P : at most V iterations $\rightarrow O(P) \leq O(V)$
- Array allocation: $O(P)$
- Second loop copies names: P iterations
 - Each iteration allocates and copies string: $O(1)$
- Second loop total: $\sum_{i=1}^P O(1) = O(P)$

Step 5 Total: $O(P) + O(P) + O(P) = O(P) \leq O(V)$

Overall Complexity:

$$\begin{aligned}
 T(V, E) &= \text{Step 1} + \text{Step 2} + \text{Step 3} + \text{Step 4} + \text{Step 5} \\
 &= O(V) + O(V) + O(V \log V) + O(EV) + O(V) \\
 &= O(EV)
 \end{aligned}$$

The dominant term is $O(EV)$ from Step 4, due to the linear `findIndex()` operation in the heap.

3 Summary Table

Operation	Time Complexity	Space
Hash Table Insert/Find	$O(1)$ avg, $O(n)$ worst	$O(n)$
MinHeap Insert/Extract	$O(\log n)$	$O(n)$
MinHeap Find Index	$O(n)$	$O(1)$
Add Vertex	$O(1)$ avg	$O(1)$
Add Edge	$O(d)$	$O(1)$
Dijkstra	$O(EV)$	$O(V)$